

HCLS Skill Development Playbook

How to Create & Update Industry Skills
in 5 Steps

Health Sciences Industry Solutions

March 2026

What This Playbook Covers

This playbook gives you the fastest path to creating or updating an HCLS industry skill for Cortex Code. It distills the full HCLS_INDUSTRY_SKILL_BEST_PRACTICES.md into actionable steps using the incubator profile and Cortex Code (CoCo) as your development environment.

Two Tracks

Track A: Create a New Skill	Track B: Update an Existing Skill
5 steps to go from zero to a working skill in the incubator	5 steps to safely modify, test, validate, and ship an update

Prerequisites

- Git access to Snowflake-Solutions/health-sciences-coco-skills-incubator
- Cortex Code installed with incubator profile:
cortex profile add health-sciences-incubator
- Snowflake account with ACCOUNTADMIN or appropriate role
- Familiarity with your domain (the skill teaches Cortex Code what YOU know)

Choose Your Archetype

Before starting, identify which pattern fits your skill. Note: many skills start as Standalone and evolve into Router + Sub-Skills as the domain is hardened and expanded.

Archetype	When to Use	Examples
Standalone	Initial version of a new domain skill, or truly single-workflow skills	FHIR (v1), Pharmacovigilance (v1), Variant Annotation, OMOP, Survival Analysis
Router + Sub-Skills	Domain has (or will grow to) multiple distinct workflows	Imaging (7 sub-skills), Clinical NLP (15 sub-skills), Pharmacov. (future: signal + dashboard)
CKE	Wrapping a Marketplace shared Cortex Search	PubMed CKE, ClinicalTrials.gov CKE
Compute-Heavy	External compute (containers, GPU, pipelines)	Nextflow nf-core, scvi-tools, Imaging ML

TIP: Start Standalone, evolve to Router. Clinical NLP started as a single SKILL.md and grew to 15 sub-skills through iterative hardening. Plan for growth from Day 1.

Track A: Create a New Skill

Step 1: Name & Scaffold

Pick your skill name using the HCLS naming convention:

```
hcls-{segment}-{domain}-{capability}
```

Segments: provider | pharma | payer | meddev | cross

Domains: cdata | imaging | genomics | dsafety | lab |
claims | risk | quality | regulatory |
postmarket | engineering | cke

Create the skill directory and bootstrap your SKILL.md:

```
cd health-sciences-coco-skills-incubator  
git checkout -b feature/hcls-{your-name}  
mkdir -p skills/hcls-{your-name}  
touch skills/hcls-{your-name}/SKILL.md
```

TIP: Use the `hcls-cross-skill-development` skill in CoCo to auto-scaffold: `cortex> "create a new HCLS skill for pharmacovigilance"`

Step 2: Build Your Skill with CoCo

Launch Cortex Code with the incubator profile and use it as your development environment. CoCo will help you write the SKILL.md, generate SQL/Python scripts, and iterate on domain knowledge.

```
# Launch CoCo with incubator profile
cortex --profile health-sciences-incubator

# Ask CoCo to help build your skill
cortex> "help me build the SKILL.md for my new
        pharmacovigilance skill"
```

What CoCo helps you write:

Part A: Frontmatter (REQUIRED)

```
---
name: hcls-pharma-dsafety-pharmacovigilance
description: "Analyze FDA FAERS data for drug safety
  signal detection. Triggers: FAERS, adverse events,
  drug safety, pharmacovigilance, signal detection."
platform_affinities:
  produces: [tables, views]
  benefits_from:
    - skill: data-governance
      when: "FAERS data has patient-level adverse events"
    - skill: developing-with-streamlit
      when: "user wants a signal detection dashboard"
---
```

Part B: Workflow Steps

The workflow is the core of your skill -- step-by-step instructions that teach CoCo how to solve the domain problem. Write it as if you're training a knowledgeable colleague:

- Setup: prerequisites, data sources, Snowflake objects needed
- Step-by-step workflow with SQL/Python code examples
- Decision points: where user input is needed (code systems, data scope, parameters)
- Stopping points: marked with MANDATORY STOPPING POINT before irreversible actions

IMPORTANT: Keep SKILL.md under 500 lines. Only include what CoCo doesn't already know -- domain expertise, not Snowflake basics.

Part C: Domain Knowledge (the real value)

Your skill's unique value is the domain knowledge that CoCo cannot infer from general training. This is what differentiates an industry skill from a generic Snowflake tutorial.

Knowledge Type	How to Add It
Domain standards & schemas	Encode in SKILL.md workflow (FHIR R4 resources, DICOM tags, FAERS file structure, OMOP CDM tables)
Reference docs (large)	Add to references/ directory (markdown, CSV) Loaded on-demand: 'Load references/faers-structure.md'
SQL scripts	Add to scripts/ directory DDL, stored procedures, pipeline setup
Python helpers	Add to scripts/ directory Data processing, API calls, parsers
Seed / reference data	Add to seed-data/ directory CSVs + setup SQL for COPY INTO
Domain PDFs	Add to skill root directory Taxonomy docs, concept schemas, standards
Data model reference	Create data-model-knowledge/ sub-skill (for router skills with 10+ tables)

Example: Clinical NLP Domain Knowledge

The clinical-nlp skill encodes: 17-table FHIR-aligned data model (245 columns), terminology seed data (154K codes across 8 code systems), extraction prompts for 6 concept categories, normalization pipelines with code-system-specific LLM guidance (RxNorm TTY hierarchy, LOINC 6-axis model, MedDRA PT level), and a 7-layer governance framework. None of this is in CoCo's general training.

Part D: Output Section (REQUIRED)

```
## Output
```

```
Describe what the skill produces: tables created,  
files generated, pipelines deployed, views, etc.
```

Step 3: Register & Integrate into Orchestrator

Your skill must be registered in the incubator's skill registry so the orchestrator can route requests to it. This is a two-part process:

Part A: Add to skills_incubator.yaml

Edit templates/skills_incubator.yaml and add your skill entry under the appropriate segment. Follow the existing format:

```
# templates/skills_incubator.yaml
skills:
  # ... existing skills ...
  hcls-pharma-dsafety-pharmacovigilance:
    description: 'FAERS signal detection'
    segment: pharma
    domain: drug-safety
    approved: false    # draft stage
```

Part B: Regenerate the Orchestrator

The orchestrator is auto-generated from the registry + Jinja2 template. Never hand-edit agents/health-sciences-incubator.md directly.

```
# Regenerate orchestrator from registry
python scripts/generate_orchestrators.py --profile incubator

# Validate orchestrator integrity (all skills referenced,
# all SKILL.md frontmatters valid, routing table complete)
python scripts/qa_validate_orchestrator.py
```

IMPORTANT: Never hand-edit agents/*.md files. Always edit the registry YAML + template, then regenerate. This prevents drift between incubator and production orchestrators.

Part C: Test with Cortex Code

```
# Test via incubator profile (full orchestrator routing)
cortex --profile health-sciences-incubator
> "analyze FAERS data for aspirin adverse events"

# Or test skill directly (bypasses orchestrator)
cortex skill add ./skills/hcls-{your-name}
> "your trigger phrase here"
```

TIP: Test BOTH paths: (1) direct skill invocation to verify the workflow works, and (2) orchestrator routing to verify trigger keywords route correctly.

Step 4: Validate

Validation ensures your skill meets the quality bar before it's shared with the team. Run through these checks systematically.

Structural Validation Checklist

- ☐ SKILL.md has name + description in frontmatter with trigger keywords
- ☐ platform_affinities declared (produces + benefits_from)
- ☐ At least one MANDATORY STOPPING POINT marker
- ☐ ## Output section present
- ☐ Under 500 lines (router can be slightly longer)
- ☐ Sub-skills have parent_skill declared (if router)
- ☐ Registered in skills_incubator.yaml
- ☐ Orchestrator regenerated and QA passes

Domain Validation

Test on real representative data for your domain:

Domain	What to Validate
Clinical NLP	Entity extraction accuracy, JSON parse rate, zero code hallucination, multi-category coverage, 50+ doc scale test
FHIR	All resource types parsed, no data loss, correct FK relationships, NDJSON + Bundle formats
Pharmacovigilance	PRR/ROR calculations match benchmarks, dedup logic, outcome stratification correctness
Imaging	DICOM tag coverage, modality-specific fields, pipeline latency at 100+ series
Genomics	ClinVar pathogenicity matches, allele frequency ranges, pipeline test profile passes
OMOP	Vocabulary mapping accuracy, measurement vs observation routing, CDM conformance

Cross-Validation Against Best Practices

Audit your skill against HCLS_INDUSTRY_SKILL_BEST_PRACTICES.md (repo root). Use the severity-based checklist:

Sev.	Check	What to Verify
CRIT	Frontmatter	name + description present, trigger keywords included
CRIT	Workflow	At least one stopping point, no chaining without approval
CRIT	Output	## Output section present in every skill and sub-skill
WARN	File size	SKILL.md under 500 lines (router can be slightly longer)
WARN	Naming	Follows hcls-{segment}-{domain}-{capability} convention
WARN	Affinities	platform_affinities declared with produces + benefits_from
WARN	Sub-skills	parent_skill declared in all sub-skill frontmatter
WARN	Evidence	skill_evidence.yaml with promotion_stage
PASS	Preflight	External dependencies have READY/MISSING/ERROR handling
PASS	Testing	Domain-specific validation tests documented/passed

Document audit results in CROSS_VALIDATION_REPORT.md inside your skill folder.

Step 5: Ship

Once validation passes, create the promotion evidence and push your skill to the incubator.

Part A: Create skill_evidence.yaml

This file tracks your skill through the promotion lifecycle (draft -> review -> staging -> production).

```
skill_name: "hcls-pharma-dsafety-pharmacovigilance"
skill_type: "customer-facing"
promotion_stage: "draft"
version: "1.0.0"
authors:
  - "sfc-gh-{your-ldap}"
validation:
  testers:
    count: 1
    names:
      - "sfc-gh-{your-ldap}"
customer_impact:
  customers:
    count: 0
    names: []
evidence_links:
  - "Validated on 50 FAERS quarterly files, PRR matches FDA benchmarks"
notes: |
  Initial draft of pharmacovigilance skill.
  Covers FAERS data loading, signal detection, and dashboard.
```

Part B: Record Customer Evidence

If you used this skill (or parts of it) in a customer demo or engagement, record it in skill_evidence.yaml. This is critical for promotion:

```
customer_impact:
  customers:
    count: 2
    names:
      - "Acme Health Systems - imaging pipeline demo"
      - "BioPharm Inc - FAERS signal detection POC"
evidence_links:
  - "Demo recording: [link to internal Sharepoint/GDrive]"
  - "Customer feedback: positive, requested follow-up"
```

TIP: Customer evidence is what moves a skill from draft to production. Even a single demo counts. Record it immediately after the engagement.

Part C: Commit and Push

```
git add skills/hcls-{your-name}/
git add templates/skills_incubator.yaml
git add agents/health-sciences-incubator.md
```



```
git commit -m "Add hcls-{your-name} skill"
git push -u origin feature/hcls-{your-name}

# Open PR to main
gh pr create --title 'Add hcls-{your-name} skill' \
  --body 'New skill for {domain}. Validated on {X} samples.'
```

Track B: Update an Existing Skill

Step 1: Branch & Understand

```
git checkout -b fix/hcls-{skill-name}-{description}

# Launch CoCo with incubator profile
cortex --profile health-sciences-incubator

# Ask CoCo to explain the current skill
cortex> "explain the workflow for hcls-provider-cdata-
        clinical-nlp"
```

Understand the skill structure before making changes:

- For standalone skills: read the workflow steps and Output section
- For router skills: read the intent table, identify which sub-skill to modify
- Check platform_affinities -- your change may need new affinities

Step 2: Make Your Changes with CoCo

Use CoCo to iterate on changes interactively:

```
# Example: adding a new sub-skill to clinical NLP
cortex> "I need to add a new extraction sub-skill for
        oncology entities to clinical-nlp"

# Example: fixing a normalization bug
cortex> "the LOINC normalization in observations is
        missing the method axis -- help me fix it"
```

Common update patterns:

Update Type	What to Change
Add a workflow step	Add step to SKILL.md workflow. Include SQL/Python examples. Add stopping point if step needs approval.
Add a sub-skill (router only)	Create new directory + SKILL.md inside parent skill. Add parent_skill to frontmatter. Add intent to router.
Fix a bug	Edit the specific step/code. Update any affected references or cross-checks across sub-skills.
Add domain knowledge	Add files to references/ or seed-data/. Update SKILL.md to reference new files.
Update platform affinities	Edit frontmatter produces/benefits_from. Regenerate orchestrator.
Add governance	Create governance/ sub-skill or add governance section to SKILL.md. Tag PHI columns.
Record customer evidence	Update skill_evidence.yaml with demo/engagement details, increment customer count.

Step 3: Re-Register & Test

If your change affected the orchestrator (new intent, new sub-skill, changed affinities), re-register and regenerate:

```
# If you changed skills_incubator.yaml or affinities:
python scripts/generate_orchestrators.py --profile incubator
python scripts/qa_validate_orchestrator.py

# Test your specific change via CoCo
cortex --profile health-sciences-incubator
> "{trigger phrase that exercises your change}"
```

IMPORTANT: If you modified a router's intent table, test ALL intents (not just the one you changed).

Step 4: Validate

Run the same validation checklist from Track A Step 4:

- Structural validation: frontmatter, stopping points, Output, file size
- Domain validation: test on representative data for the modified area
- Cross-validation: audit against best practices if significant changes
- Regression: verify unchanged workflows still work after your edit

TIP: For Clinical NLP, the validation sweep covered 7 code systems x 6 normalization sub-skills = 32 fixes. Any change to one sub-skill should be cross-checked against related sub-skills.

Step 5: Ship

Update evidence and push:

- Update skill_evidence.yaml: increment version, add testers, add customer evidence
- Update CROSS_VALIDATION_REPORT.md if you did a re-audit

```
git add -A
git commit -m "Update hcls-{skill-name}: {description}"
git push -u origin fix/hcls-{skill-name}-{description}

# Open PR to main
gh pr create --title 'Update hcls-{skill-name}: {desc}'
```

Quick Reference: Router Skill Anatomy

Router skills are the most complex archetype. Here's the Clinical NLP skill as a concrete example:

```
skills/hcls-provider-cdata-clinical-nlp/
  SKILL.md                                # Router: 17 intents, preference gate
  skill_evidence.yaml                     # Promotion: draft, v1.0.0
  CROSS_VALIDATION_REPORT.md             # Audit: 0 critical, 9 warnings fixed
  *.pdf                                  # Domain taxonomy + concept schema
  data-model-knowledge/                   # 245-row model ref + Cortex Search
    SKILL.md / references/ / scripts/ / seed-data/
  extraction-conditions-diagnostics/SKILL.md
  extraction-therapeutics/SKILL.md
  extraction-observations/SKILL.md
  extraction-patient-context/SKILL.md
  extraction-oncology/SKILL.md
  extraction-safety-care-planning/SKILL.md
  normalization-conditions-diagnostics/SKILL.md
  normalization-therapeutics/SKILL.md
  normalization-observations/SKILL.md
  normalization-patient-context/SKILL.md
  normalization-oncology/SKILL.md
  normalization-safety-care-planning/SKILL.md
  governance/SKILL.md                     # 7-layer PHI governance
  pipeline-implementation/SKILL.md        # DTs + SP pipeline
```

Router SKILL.md Template

```
---
name: hcls-{name}
description: "... Triggers: ..."
platform_affinities:
  produces: [tables, views, ...]
  benefits_from:
    - skill: data-governance
      when: "output contains PHI"
---
# Title
## Preflight Check (probe data model knowledge svc)
## Intent Detection
| Intent | Triggers | Load |
| EXTRACT_CONDITIONS | 'extract conditions' | extraction-.../SKILL.md |
| NORMALIZE_ALL | 'normalize all' | (iterates all 6) |
| GOVERNANCE | 'governance, PHI' | governance/SKILL.md |
## Step 0: Data Model Knowledge (conditional on preflight)
## Stopping Points
## Output
```

Sub-Skill SKILL.md Template

```
---
name: extraction-conditions-diagnostics
description: "Extract conditions from clinical notes..."
parent_skill: hcls-provider-cdata-clinical-nlp
---
# Conditions & Diagnostics Extraction
## Scope (entity types -> target tables)
## Engine Strategy (Cortex AI + regex supplements)
## Extraction Prompt (structured JSON output schema)
## Post-Processing (LATERAL FLATTEN -> INSERT)
## Stopping Points
## Output
```

Quick Reference: Governance Patterns

Choose the right governance approach based on your data sensitivity:

Example: Clinical NLP Governance Sub-Skill (7 layers)

- L1: Tag-based PHI classification (16 PHI columns across 12 tables)
- L2: Column masking policies (tag-based, one per data type)
- L3: Row-access policies (department-based + patient-attribution)
- L4: 3-tier database roles (READER < ANALYST < ADMIN)
- L5: Cortex AI guardrails (secure views excluding PHI)
- L6: ML feature views (de-identified, hashed patient IDs)
- L7: Audit queries (ACCESS_HISTORY + QUERY_HISTORY)

Data Type	Governance Level	What to Implement
PHI (patient names, MRNs, dates, notes)	High (dedicated sub-skill)	Tag-based PHI classification, column masking, row-access policies, 3-tier roles, AI guardrails, ML de-identification, audit queries
Patient-level (claims, adverse events, encounters)	Medium (section in SKILL.md)	Column masking on identifiers, role-based access, platform affinity to data-governance
Identifiable genomic data (sample-to-patient)	Medium (section in SKILL.md)	De-identification of mapping tables, secure compute for variant data, access policies
Aggregate / de-identified data only	Low (affinity only)	Declare platform affinity to data-governance. No masking needed for aggregate data.
Reference data (CKEs, ontologies, public datasets)	None	No governance needed. Read-only shared services with no patient data.

Cheat Sheet: Commands & Examples

Skill Development with CoCo

```
# Scaffold new skill
cortex> "create a new HCLS skill for drug safety"

# Iterate on an existing skill
cortex> "help me add a FAERS quarterly data loader
        to pharmacovigilance"

# Test skill via orchestrator routing
cortex --profile health-sciences-incubator
> "extract conditions from these clinical notes"
> "normalize all entities to ICD-10-CM and SNOMED CT"
> "set up PHI governance for clinical NLP tables"
```

Orchestrator Management

```
# Regenerate after registry changes
python scripts/generate_orchestrators.py --profile both

# Validate integrity
python scripts/qa_validate_orchestrator.py
```

Git Workflow

```
# New skill:  feature/hcls-{name}
# Bug fix:   fix/hcls-{name}-{desc}
# Docs:      docs/{desc}
# Refactor:  refactor/{desc}

git push -u origin {branch}
gh pr create --title '{title}' --body '{description}'
```

Validation Queries (Clinical NLP Example)

```
# Extraction coverage
SELECT COUNT(*) total, COUNT(DISTINCT provenance_document_id) docs
FROM CONDITION WHERE source = 'GENAI_NLP_NOTE';

# Normalization rate
SELECT COUNT(*) total, COUNT(code) coded,
       ROUND(COUNT(code)*100.0/NULLIF(COUNT(*),0),1) pct
FROM CONDITION WHERE source = 'GENAI_NLP_NOTE';

# Zero hallucination check
SELECT c.display, c.code, c.code_system FROM CONDITION c
```

HCLS Skill Development Playbook
WHERE c.code IS NOT NULL AND c.source = 'GENAI_NLP_NOTE'
AND NOT EXISTS (SELECT 1 FROM CONCEPT_DIMENSION cd
WHERE cd.code = c.code); -- Expected: 0 rows

Skill Integrity Queries

```
# Count all skills
ls -d skills/hcls-*/ | wc -l

# SKILL.md line counts (should be <500)
wc -l skills/*/SKILL.md | sort -rn

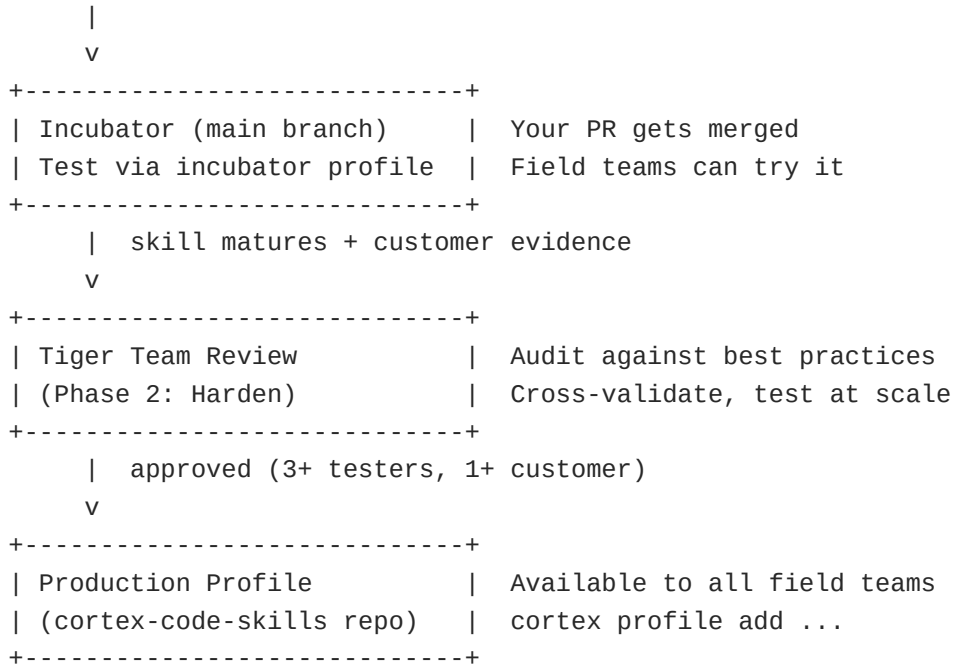
# Verify all have required sections
grep -l '^name:' skills/*/SKILL.md | wc -l
grep -rl '^## Output' skills/*/SKILL.md | wc -l

# Check sub-skill parent_skill references
grep -r 'parent_skill:' skills/*/SKILL.md
```


What's Next?

You've created or updated a skill. Here's what happens next in the lifecycle:

YOU ARE HERE



Key Resources

- HCLS_INDUSTRY_SKILL_BEST_PRACTICES.md -- Full best practices guide (repo root)
- SKILL_BEST_PRACTICES.md -- Platform skill best practices (cortex-code-skills repo)
- documentation/ISF-*.pdf -- Industry Solutions Framework lifecycle
- templates/skills_incubator.yaml -- Skill registry (source of truth)
- agents/health-sciences-incubator.md -- Orchestrator (auto-generated, never hand-edit)

Getting Help

- Use hcls-cross-skill-development skill in CoCo for guided scaffolding
- File an issue on the incubator repo for questions or feature requests
- Tag the Tiger Team for promotion reviews
- Ask CoCo directly: 'help me improve my HCLS skill'